

UV for Beginners

Student Study Guide

Learn • Practice • Build

A hands-on guide to creating, running and managing Python projects with uv.

By the end, you will be able to:

- Create a Python project with `uv init`
- Explore a uv project and find `pyproject.toml`
- Explain how a uv project is run
- Create a new Python file and register it as a script in `pyproject.toml`
- Run your own scripts with `uv run`
- Add and remove packages and see what changes in `pyproject.toml`

How to Use This Guide

This is a study guide, not a command list. In every section you will **read** a short explanation, **do** the step yourself, and **look** at what happens. If your result does not match, re-read the step and try again.

GOLDEN RULE

Don't just copy commands. Type them yourself and see what they do.

The boxes you will see

Box	What it asks you to do
LOOK FOR	Stop and observe. Open a file or read the output and find something specific.
REMEMBER	A key idea worth keeping.
WATCH OUT	A common mistake to avoid.
TRY IT YOURSELF	A short exercise. No copying, use what you just learned.

Learning Path

#	Topic	Goal
1	What is uv?	Understand the problem uv solves
2	Create a project	Use <code>uv init</code> and read the folder structure
3	Explore <code>pyproject.toml</code>	Find the name, dependencies and scripts
4	Run the project	See how the project runs with <code>uv run</code>
5	Add your own script	Create a new file and register it in <code>pyproject.toml</code>
6	<code>main()</code> and <code>__name__</code>	Understand the pattern in your code
7	Add and remove packages	Use <code>uv add</code> and <code>uv remove</code>
8	Development dependencies	Use <code>uv add --dev</code>
9	The uv workflow	Put the steps together
10	Read the error	Fix the most common mistakes
11	Practice project	Build a small app on your own

1. What Is uv?

Python is the programming language. uv is a tool that helps us manage Python projects.

Python	uv
The programming language. It runs your code.	The project manager. It creates projects, manages environments, packages and dependencies, and runs your project.

REMEMBER

Think of it this way: Python helps you write the program. uv helps you organize and manage the project around that program.

Check that uv is installed

```
uv --version
```

LOOK FOR

You should see a version number such as `uv 0.12.3`. This guide needs **uv 0.12 or newer**. If you see “command not found”, tell your instructor before continuing.

Quick Check

1. Is uv a programming language?
2. What is Python used for?
3. What does uv help us manage?

2. Create Your First Project

A project is a folder containing the code and files needed for an application. uv creates that folder and its starting files for you.

Step 1 — Create the project

```
uv init my-first-project
```

WHAT KIND OF PROJECT IS THIS?

uv created a **packaged** project: your code lives inside a package folder, and the project can define named commands (called **scripts**) in `pyproject.toml`. You will create your own script in Section 5.

WATCH OUT

If uv gave you a `main.py` file instead of a `src` folder, your uv is older than 0.12. Update uv, or create the project with `uv init --package my-first-project` instead.

Step 2 — Enter the project folder

```
cd my-first-project
```

Step 3 — Look at what uv created

To list the files in the folder:

System	Command
Windows	<code>dir</code>
Mac / Linux	<code>ls -a</code>
Any system	Open the folder in your code editor (for example VS Code)

You should see something like this:

```
my-first-project/
├── .gitignore           ← files Git should ignore
├── .python-version      ← which Python version to use
├── README.md           ← project notes
├── pyproject.toml       ← project information (open this next!)
├── src/
│   ├── my_first_project/ ← the folder that holds your code
│   └── __init__.py      ← your first code file
```

LOOK FOR

1. There is no `.venv` folder and no `uv.lock` file yet. `uv` creates them the first time you run the project (Section 4).
2. The project name has hyphens (`my-first-project`) but the code folder has underscores (`my_first_project`). Python names can't contain hyphens, so `uv` converts them for you. If you typed underscores when creating the project, `uv` still names it with hyphens.
3. You may also see a `.git` folder if Git is installed on your computer. That is fine.

REMEMBER

`uv init` means “create a new project”.

TRY IT YOURSELF

Go up one folder with `cd ..`, then create a second project:

```
uv init practice-project
```

Find its code folder inside `src`. What is the folder called?

When you finish, go back with `cd ..` and then `cd my-first-project`.

3. Explore pyproject.toml

`pyproject.toml` is the most important file in a Python project. Think of it as the project's ID card and instruction sheet. It tells uv the project's name, which Python it needs, which packages it depends on, and **how the project is run**.

Open `pyproject.toml` in your editor. It looks like this:

```
[project]
name = "my-first-project"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []

[project.scripts]
my-first-project = "my_first_project:main"

[build-system]
requires = ["uv_build>=0.12.3,<0.13.0"]
build-backend = "uv_build"
```

Some numbers (like the Python version) may be different on your computer. That is fine.

Section	What it tells us
[project]	The project's name, version, description and the Python version it needs
dependencies	The packages the project needs. It is empty for now.
[project.scripts]	Named commands. This is how the project is run. We will focus on this next.
[build-system]	How uv packages the project. uv sets this up, so leave it alone.

The file format is called TOML. Section names go in `[square brackets]`, settings look like `key = "value"`, and lists go in `[ ]`.

LOOK FOR

Find these four things in your own `pyproject.toml`:

1. The project name
2. The Python version it needs (`requires-python`)
3. The script name (on the left of the `=` sign)
4. What the script points to (on the right of the `=` sign)

4. Run the Project

Now the big question: **how is this project run?** Look at the line under `[project.scripts]`:

```
my-first-project = "my_first_project:main"
```

Read it like a sentence: “When someone runs the command `my-first-project`, call the function `main` inside the package `my_first_project`.”

So where is `main`? Open `src/my_first_project/__init__.py`:

```
def main() -> None:
    print("Hello from my-first-project!")
```

Run it

```
uv run my-first-project
```

Expected output:

```
Hello from my-first-project!
```

`uv` may also print a few lines about creating an environment or installing the project. That is normal.

REMEMBER

What does `uv run` mean? It means: “`uv`, prepare this project’s environment if needed, then run this command inside it.”

LOOK FOR: WHAT CHANGED?

List your files again. Two new things appeared. `uv` created them during the first run:

```
my-first-project/
├── .venv/           ← NEW: the project’s private environment
├── uv.lock          ← NEW: the exact package versions uv chose
├── pyproject.toml
└── src/my_first_project/
```

`.venv` is a private environment for this one project. Packages you add are installed here, so one project’s packages never mix with another’s. `uv.lock` records the exact versions installed. `uv` manages both, so never edit them by hand.

Synchronize the environment

```
uv sync
```

`uv sync` makes `.venv` match what `pyproject.toml` describes. `uv run` does this automatically, so you will not type it often. It is useful after you edit dependencies by hand or download someone else’s project.

TRY IT YOURSELF

Change the message in `__init__.py` to include your name, then run `uv run my-first-project` again. Change it once more and run again. Notice that you don't need to reinstall anything.

5. Add Your Own Script

`uv` gave us one script. Now **you** will build another: create a new file, then register it in `pyproject.toml` so you can run it with `uv run`.

Step 1 — Create a new file

In your editor, right-click the `my_first_project` folder (the one that contains `__init__.py`), choose **New File**, and name it `greeter.py`. It must sit next to `__init__.py`:

```
src/
├── my_first_project/
│   ├── __init__.py
│   └── greeter.py    ← your new file
```

Step 2 — Write the code

```
def main() -> None:
    name = input("What is your name? ")
    print(f"Hello, {name}! Welcome to uv.")
```

Step 3 — Register the script in `pyproject.toml`

Open `pyproject.toml`. Under `[project.scripts]`, add a new line below the existing one:

```
[project.scripts]
my-first-project = "my_first_project:main"
greet = "my_first_project.greeter:main"
```

Here is how to read the new line:

Part	What it means
<code>greet</code>	The command name. You choose it. It is what you type after <code>uv run</code> .
<code>my_first_project</code>	The package folder inside <code>src</code>
<code>.</code> (dot)	Separates the folder from the file
<code>greeter</code>	Your file name, without <code>.py</code>
<code>:</code> (colon)	Separates the file from the function
<code>main</code>	The function <code>uv</code> should call

WATCH OUT

Put quotes around the right-hand side. Use a **dot** between folder and file and a **colon** before the function name. Spell every name exactly like your file and function.

Step 4 — Run it

```
uv run greet
```

Expected output (type your own name):

```
What is your name? Ali
Hello, Ali! Welcome to uv.
```

The first time after editing `pyproject.toml`, uv may print “Building” and “Installed 1 package”. That is uv updating your project.

REMEMBER

Two places, one job. The code lives in a `.py` file. `pyproject.toml` gives that code a command name.

TRY IT YOURSELF

Create a second file, `goodbye.py`, next to `greeter.py`, with a `main()` function that prints a goodbye message. Register it as `bye`, then run it.

6. Understanding `main()` and `__name__`

You wrote a `main()` function in every file. This pattern is normal Python. **It is not a special uv feature.** You will often see this at the bottom of Python files:

```
if __name__ == "__main__":
    main()
```

`main()` holds the main work of the program. The `if` line means, roughly: “If this file is being run **directly**, call `main()`.”

See the difference

A. Run your file directly, as a module. The dotted path is the same one you used in `pyproject.toml`:

```
uv run python -m my_first_project.greeter
```

Nothing happens. The file defines `main()` but never calls it.

B. Add these two lines at the bottom of `greeter.py`, then run the same command again:

```
if __name__ == "__main__":  
    main()
```

Now it works. **C.** Finally run `uv run greet` again.

LOOK FOR

In step C, did the greeting print once or twice? Why do you think that is?

Explanation: `uv run greet` calls `main()` for you. Your file is loaded as a module, not run directly, so the `if` block is skipped and the greeting prints only once.

7. Add and Remove a Package

Sometimes Python's standard library is not enough. We can add an outside package to the project.

Add a package

```
uv add requests
```

`uv` downloads the package, installs it in `.venv`, and records it in `pyproject.toml`.

LOOK FOR

Open `pyproject.toml`. The `dependencies` line has changed. It should look like this (your version number may differ):

```
dependencies = [  
    "requests>=2.34.2",  
]
```

Use it in a new script

Create `webcheck.py` next to your other files:

```
import requests  
  
def main() -> None:  
    response = requests.get("https://example.com")  
    print(response.status_code)
```

Register it under `[project.scripts]` with the name `check`, then run it (you need an internet connection):

```
check = "my_first_project.webcheck:main"  
  
uv run check
```

You should see 200, which means the website answered.

Remove a package

```
uv remove requests
```

dependencies in `pyproject.toml` goes back to `[]`.

Command	Meaning
<code>uv add package</code>	The project needs this package. Install it and record it.
<code>uv remove package</code>	The project no longer needs it. Uninstall it and remove the record.

TRY IT YOURSELF

Predict first: you removed `requests`, but `webcheck.py` still uses it. What will happen if you run `uv run check`?

Now run it. Read the **last line** of the error. Then clean up: delete `webcheck.py` and delete the check line from `pyproject.toml`.

8. Development Dependencies

Some packages help us **build or test** a project but are not needed when the project simply runs. These are development dependencies.

```
uv add --dev pytest
```

The `--dev` option tells uv this is a development tool. It is recorded in its own section:

```
[dependency-groups]
dev = [
    "pytest>=9.1.1",
]
```

Kind	Command	Recorded in <code>pyproject.toml</code> under
Needed to run the project	<code>uv add package</code>	<code>dependencies</code>
Only for development or testing	<code>uv add --dev package</code>	<code>[dependency-groups]</code> then <code>dev</code>

9. The uv Workflow to Remember

Step	What you do	Command or action
Create	Start a new project	<code>uv init project-name</code>
Enter	Move into the folder	<code>cd project-name</code>
Inspect	Find how it runs	Open <code>pyproject.toml</code> , read <code>[project.scripts]</code>
Run	Run a script	<code>uv run script-name</code>
Extend	Add a feature	New file in <code>src/project_name/</code> , then a new line in <code>[project.scripts]</code>
Need a package?	Add it	<code>uv add package-name</code>
Edited dependencies by hand?	Update the environment	<code>uv sync</code>

REMEMBER

The goal is not to memorize dozens of commands. The goal is to get comfortable with this workflow.

10. Read the Error

Everyone makes mistakes. Here are the four you are most likely to meet, with the real messages uv and Python show. **Read the last line of an error first.** It usually names the problem.

What went wrong	The error's last line	How to fix it
Typed a script name that doesn't exist: <code>uv run gret</code>	<code>error: Failed to spawn: gret</code>	The name must match one in <code>[project.scripts]</code> . Check the spelling.
Misspelled the file name in <code>pyproject.toml</code>	<code>ModuleNotFoundError: No module named 'my_first_project.greter'</code>	Make the name after the dot match your <code>.py</code> file.
Misspelled the function name after the colon	<code>ImportError: cannot import name 'mian'</code>	Make the name after the colon match your function.
Removed a package your code still imports	<code>ModuleNotFoundError: No module named 'requests'</code>	Add it back with <code>uv add</code> , or remove the code that uses it.

TRY IT YOURSELF

Break your `greet` script on purpose: change `greeter` to `greter` in `pyproject.toml` and run `uv run greet`. Read the error, match it to the table, then fix it.

11. Final Practice Project

Build a Personal Introduction App

This time there are no step-by-step instructions. Use everything you have learned. Create a **new** project called `intro-app` and build a program that:

- Was created with `uv init intro-app`
- Lives in a new file, `src/intro_app/introduce.py`
- Uses a `main()` function
- Asks the user's name, age and favorite programming language
- Prints a friendly introduction
- Is registered in `pyproject.toml` as a script named `intro`
- Runs with `uv run intro`

Example run

```
What is your name? Ali
How old are you? 20
What is your favorite language? Python

Hello Ali!
You are 20 years old.
You like Python.
```

HINTS (ONLY IF YOU GET STUCK)

1. The project name has a hyphen (`intro-app`) but the code folder has an underscore (`intro_app`).
2. The new file must be inside `src/intro_app/`, next to `__init__.py`.
3. The script line looks like: `name = "folder.file:function"`

Challenge — Level Up

Add one more question of your choice, and make the output clean and friendly.

Bonus: run `uv add rich`, then put `from rich import print` at the top of your file. What changes when you print?

12. Knowledge Check

Answer these in your own words to test your understanding. If you get stuck, go back to the section.

1. What problem does uv help solve?
2. Which command creates a new uv project?
3. Which file holds the project's name, dependencies and scripts?
4. Which section of `pyproject.toml` defines how the project is run?
5. In `"my_first_project.greeter:main"`, what are `my_first_project`, `greeter` and `main`?
6. How do you run a script named `greet`?
7. When are `.venv` and `uv.lock` created?
8. How do you add a package? How do you remove one?
9. What does `--dev` mean, and where is the package recorded?
10. What does `if __name__ == "__main__":` generally mean?

13. uv Quick Reference

Command	Use it when...
<code>uv --version</code>	You want to check that uv is installed
<code>uv init project</code>	You want to create a new project
<code>uv run script-name</code>	You want to run a script from <code>[project.scripts]</code>
<code>uv run python -m package.module</code>	You want to run one file directly
<code>uv sync</code>	You want <code>.venv</code> to match <code>pyproject.toml</code>
<code>uv add package</code>	Your project needs a package
<code>uv remove package</code>	Your project no longer needs a package
<code>uv add --dev package</code>	You need a development-only package

GOLDEN RULE

Final reminder: Type the commands yourself. Make mistakes. Read the error. Try again. That is how you become comfortable with your tools.